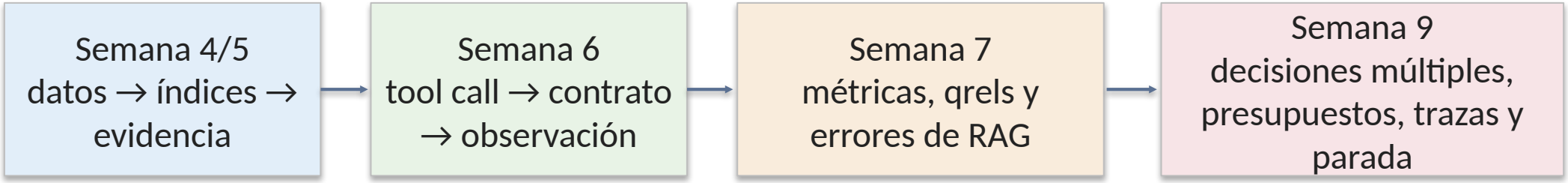
1. De workflows a agentes

Semana 9: Workflows, agentes, ReAct, planificación y criterios de parada

Semana 9 conserva el foco experimental de CC-0F4: comparar sistemas sobre el mismo benchmark y separar con claridad qué cambia en el controlador de ejecución.



Pregunta de la semana: ¿qué ventajas y riesgos introduce permitir que un LLM seleccione acciones respecto de ejecutar una secuencia fija o una política de reglas?

Lectura: La semana 9 introduce un controlador con decisiones auditables, no memoria persistente; esa frontera se reserva para la semana 10.

2. Workflow frente a agent loop

La diferencia central está en el origen de las decisiones y en cómo se audita la ejecución

$$\text{workflow: decisions in code} \qquad \text{agent: decisions from } \pi_\theta$$

WORKFLOW

La ruta se fija en código. Las ramas posibles se definen antes de ejecutar.

CONTROLADOR HEURÍSTICO

El sistema observa y cambia de ruta, pero solo siguiendo reglas explícitas.

AGENTE LLM

La política propone acciones usando el estado y las observaciones acumuladas.

Más flexibilidad no implica automáticamente mejor evidencia ni menor costo.

Lectura: La complejidad adicional debe justificarse experimentalmente con éxito, trazas, costo y análisis de fallos.

3. Estado, política y transición

El controlador se formaliza como estado transitorio, política de decisión y observación de herramientas

Estado transitorio s_t

observaciones, citas vistas, contadores, límites y razón de parada pendiente

$$s_{t+1} = F(s_t, a_t, o_{t+1})$$

$$a_t \sim \pi_\theta(\cdot \mid s_t)$$

Política pi_theta

puede ser un script, una heurística o un modelo local que selecciona acciones

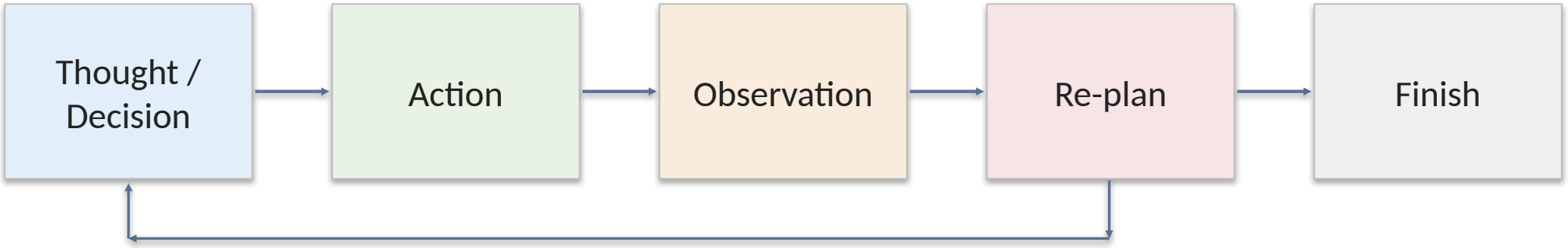
Semana 9 trabaja únicamente con estado de una ejecución. No crea working memory, episodic memory ni semantic memory entre sesiones.

Lectura: La formalización distingue claramente el bucle de ejecución de la memoria persistente que se estudiará después.

4. ReAct y el ciclo decisión–acción–observación

ReAct no es “usar un LLM”, sino intercalar razonamiento operativo, acción y observación

$$\text{Thought} \rightarrow \text{Action} \rightarrow \text{Observation} \rightarrow \cdots \rightarrow \text{Finish}$$



La implementación base mantiene la autoridad en el controlador: valida la propuesta, verifica la allowlist y registra si la observación produce progreso o no.

Lectura: ReAct exige decidir con observaciones reales; un replay escrito por el programador demuestra el controlador, no la autonomía del modelo.

5. Cinco patrones de workflow

La semana 9 contrasta patrones deterministas antes de entrar al agente con herramientas

Prompt chaining

salida validada
alimenta el siguiente paso

Routing

una categoría
define la ruta

Parallelization

ramas independientes
se ejecutan en paralelo

Evaluator–optimizer

un ciclo revisa
y corrige hasta aceptar

Orchestrator–workers

coordina subtareas
pero sigue siendo código

Un patrón con ramas no se convierte en agente solo por tener más cajas o más herramientas.

Lectura: Esta clasificación prepara la comparación entre A0/A1/B y el controlador del agente de la misma semana.

6. Contratos, herramientas y criterios de parada

El valor de la semana 9 está en validar cada acción propuesta antes de ejecutarla

Allowlist

search_documents
read_passage
finish

Todo lo demás se rechaza.

Presupuestos

max_steps
max_tool_calls
max_invalid_actions
deadline_seconds
max_total_tokens

Observaciones

resultado correcto
error de herramienta
finish inválido
acción repetida
sin progreso

$$\text{stop} \in \{\text{completed}, \text{max_steps}, \text{max_tool_calls}, \text{deadline}, \text{invalid_action}\}$$

Lectura: El controlador impone límites explícitos y devuelve una razón de parada auditable; esto no depende de lo que prometa el prompt del modelo.

7. La cobertura léxica como señal de control

B no usa un LLM: utiliza una regla medible sobre el primer pasaje recuperado

$$c(q,d) = \frac{|\mathrm{tok}(q) \cap \mathrm{tok}(d)|}{|\mathrm{tok}(q)|}$$

Interpretación

q: términos de la consulta
d: primer pasaje recuperado
0 ≤ c(q,d) ≤ 1

Ejemplo

q = {what, year, was, …}
5 términos coinciden de 8

c(q,d)=5/8=0.625

Uso en el sistema

si la cobertura es baja, el sistema amplía contexto antes de terminar.

Lectura: La cobertura léxica es una señal de control simple y auditable; no pretende modelar relevancia semántica completa.

8. La regla adaptativa B

La decisión de ampliar a top-3 se fija antes de observar los resultados y no usa qrels

A0

una sola llamada
search(top_k=1)

A1

una sola llamada
search(top_k=3)

$$k(q) = \begin{cases} 1, & c(q, d_1) \geq 0.45 \\ 3, & c(q, d_1) < 0.45 \end{cases}$$

El umbral 0.45 no se ajusta post hoc. Se declara antes de comparar y se mantiene fijo para las 16 consultas.

B

1) search(top_k=1)
2) medir cobertura
3) si c<0.45,
search(top_k=3)
4) como máximo dos
llamadas

Lectura: B es un controlador adaptativo basado en reglas; su función es separar decidir de ejecutar.

9. Benchmark auténtico de la semana 9

La edición actual no reutiliza el corpus de práctica de la semana 4; usa una muestra real de SQuAD 1.1 dev

8 artículos reales de Wikipedia

32 pasajes originales

16 preguntas redactadas por personas

qrels por pasaje con respuesta

Los tres sistemas offline utilizan el mismo BM25 local, la misma tokenización y el mismo ranking. Cambia únicamente la política de control.

No se mide exactitud semántica de la respuesta ni costo de API del LLM en este benchmark offline.

Lectura: El uso de un benchmark real y pequeño impide confundir el experimento con una demostración ad hoc construida con datos sintéticos.

10. Métrica operativa: evidence_hit

La medida principal solo verifica si alguna cita coincide con al menos un pasaje relevante

$$\mathrm{EvidenceHit}(q) = \mathbb{1}[R_q \cap C_q \neq \varnothing]$$

R_q

conjunto de pasajes relevantes para la consulta q

→

C_q

conjunto de pasajes citados por el sistema

→

Salida

1 si hay intersección
0 en caso contrario

EvidenceHit no evalúa fidelidad completa de la respuesta, grounding de cada afirmación ni calidad del agente LLM.

Lectura: La métrica se diseñó para aislar la política de control sin introducir una evaluación semántica más amplia.

11. Invariante del diseño experimental

Con el mismo ranking fijo, B jamás puede superar a A1 en evidence_hit

$$\mathrm{Hit}(A0) \leq \mathrm{Hit}(B) \leq \mathrm{Hit}(A1)$$

A0 cita un subconjunto del ranking top-1

→

B puede quedarse con top-1 o ampliar a top-3

→

A1 siempre observa el ranking top-3

La posible ventaja de B es usar menos contexto cuando el primer pasaje ya ofrece cobertura suficiente.

Lectura: Este invariante evita una conclusión incorrecta: decidir más no implica recuperar evidencia nueva si el ranking subyacente no cambia.

12. Contexto, costo y token proxy

La semana 9 mide una aproximación léxica del tamaño de contexto, no tokens reales del modelo

$$\text{context_token_proxy}(q) = \sum_{d \in C_q} |\text{tok}(d)|$$

A0	A1	B
menor costo de contexto una sola llamada	mayor cobertura fija una sola llamada	trade-off entre contexto y llamadas

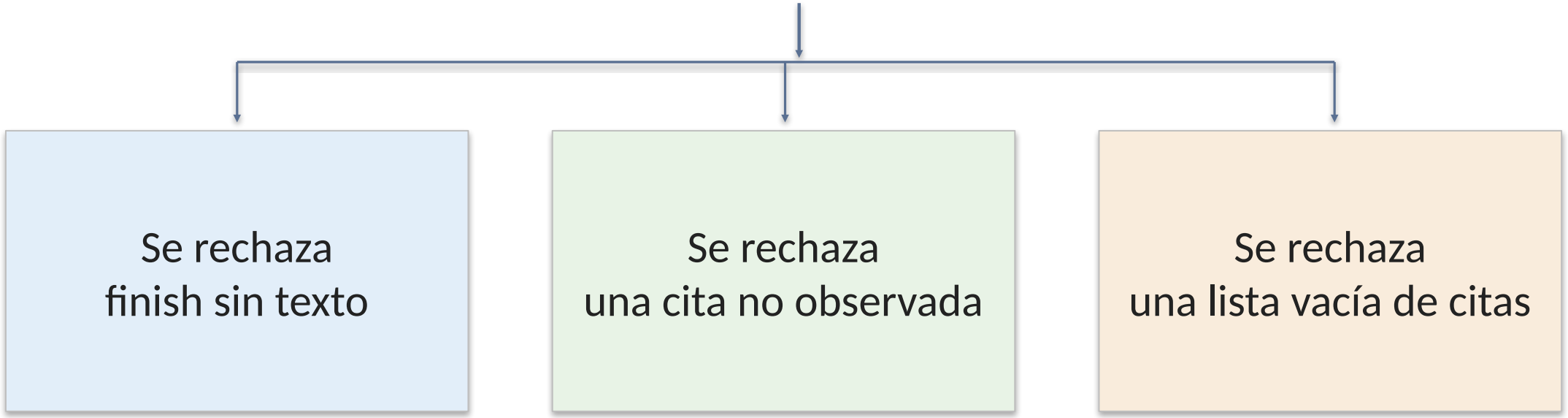
Latencia BM25 local y context_token_proxy son medidas operativas del experimento, no equivalen a costo API ni a latencia total de un LLM real.

Lectura: El curso separa los costos observables del benchmark offline de los costos de un agente con un modelo real.

13. Validez de finish y grounding mínimo

El controlador solo acepta una finalización si la respuesta tiene texto y cita evidencia observada

$$\text{finish}(y, \mathcal{C}) \Rightarrow y \neq \emptyset \ \wedge \ \mathcal{C} \subseteq \mathcal{O}_{\text{seen}} \ \wedge \ |\mathcal{C}| \geq 1$$



Grounding mínimo no significa verdad completa: solo garantiza que la salida referencia evidencia que el sistema sí vio.

Lectura: El criterio operacional de finish permite discutir la diferencia entre salida aceptada y respuesta semánticamente correcta.

14. Acciones repetidas y límites de seguridad

Una heurística simple detecta bucles sin progreso antes de agotar el presupuesto completo

$$\sigma_t = (\text{action}_t,\ \text{arguments}_t)$$

$$\sigma_t = \sigma_{t-1} \ \wedge\ \text{no progress} \ \Rightarrow\ \text{repeated_action}$$

Ejemplo de rechazo

run_shell no está en la allowlist

Ejemplo de recuperación

read_passage inválido → nueva búsqueda

Ejemplo de ciclo

search idéntico dos veces

La detección de repetición es deliberadamente simple: compara acción y argumentos, no equivalencia semántica profunda.

Lectura: El objetivo es mostrar cómo un controlador puede defenderse de políticas defectuosas sin ejecutar comandos arbitrarios.

15. Experimento principal de la semana 9

Mismo benchmark, mismo retriever, misma métrica operativa; cambia la política de control

Condición A0 workflow_fixed_top1	Condición A1 workflow_fixed_top3	Condición B adaptive_rule_based	Replay / LLM demo fuera del benchmark offline

Métricas principales

evidence_hit_rate
mean_tool_calls
mean_steps
mean_context_token_proxy
mean_latency_ms

Lectura: El experimento pareado debe analizarse junto con sus amenazas a la validez: muestra pequeña, qrels no exhaustivos y ausencia de evaluación semántica.

16. Diferencias pareadas y lectura crítica del resultado

Las métricas medias permiten resumir el benchmark, pero no sustituyen el análisis por consulta y por traza

$$\bar{M}(P) = \frac{1}{n}\sum_{i=1}^{n} M_i(P)$$

$$d_i = M_i(B) - M_i(A)$$

Promedio por política

resume el comportamiento global del benchmark.

Diferencia por consulta

permite estudiar cuándo B ahorra contexto o falla.

Trazas

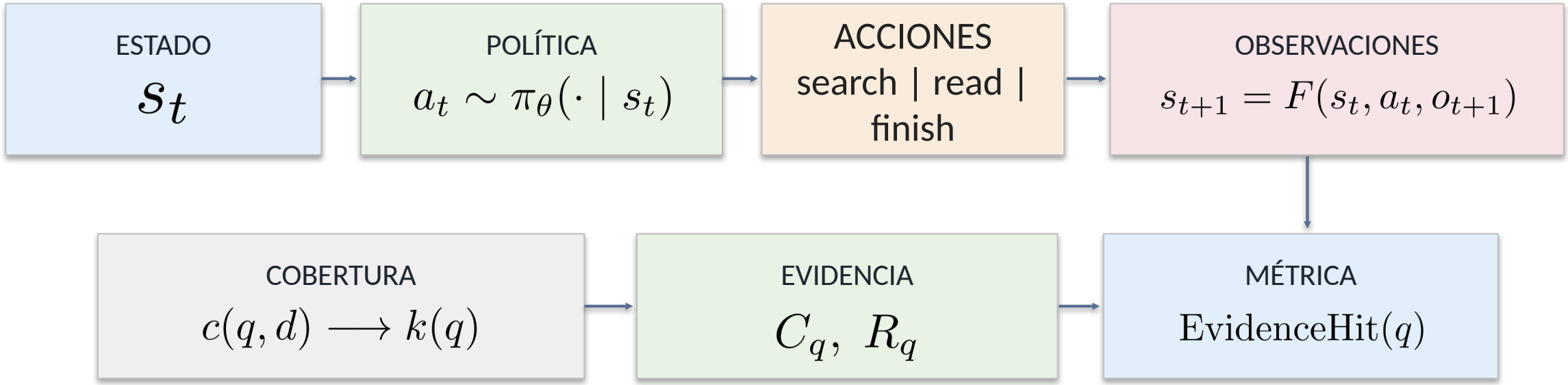
explican por qué se produjo el resultado.

El curso no debe reducir la interpretación a ganó una media; la traza es parte del resultado experimental.

Lectura: Esta transición prepara el harness de evaluación más amplio que se trabajará en la semana 13.

17. Mapa final de ecuaciones – Semana 9

Del estado del agente a la evidencia recuperada y la razón de parada



mejor control ≠ mejor evidencia automáticamente | medir → analizar trazas → limitar la conclusión

Lectura: Cierre: la semana 9 formaliza y experimenta el controlador; la semana 10 añadirá memoria sobre esta misma base.